

MapReduce Pipeline Documentation

Step 1: Input Data Upload to S3

We downloaded the Hamlet text file and uploaded it to an S3 bucket under the `input/` prefix.

S3 Input Path

```
s3://mapreduce-nithish-hw4/input/hamlet.txt
```

This file served as the input to the splitter task.

```
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4> aws s3 ls s3://mapreduce-nithish-hw4/input/
2026-02-08 20:24:55      0
2026-02-08 20:25:14 162881 hamlet.txt
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4>
```

- S3 bucket showing `input/hamlet.txt`

Step 2: ECS Cluster Setup

We created an ECS cluster named `mapreduce-cluster` using the Fargate launch type, allowing us to run containers without managing EC2 instances.

The cluster was used to run all tasks in the pipeline.

Clusters (2) Info

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Capacity provider strategy
default	0	No tasks running	0 EC2	Ⓢ Default	No default found
mapreduce-cluster	0	No tasks running	0 EC2	Ⓢ Default	No default found

- ECS clusters page showing `mapreduce-cluster`

Step 3: Splitter Task Execution

The splitter task was invoked via HTTP with the S3 input file URL and a chunk count of 3.

Request

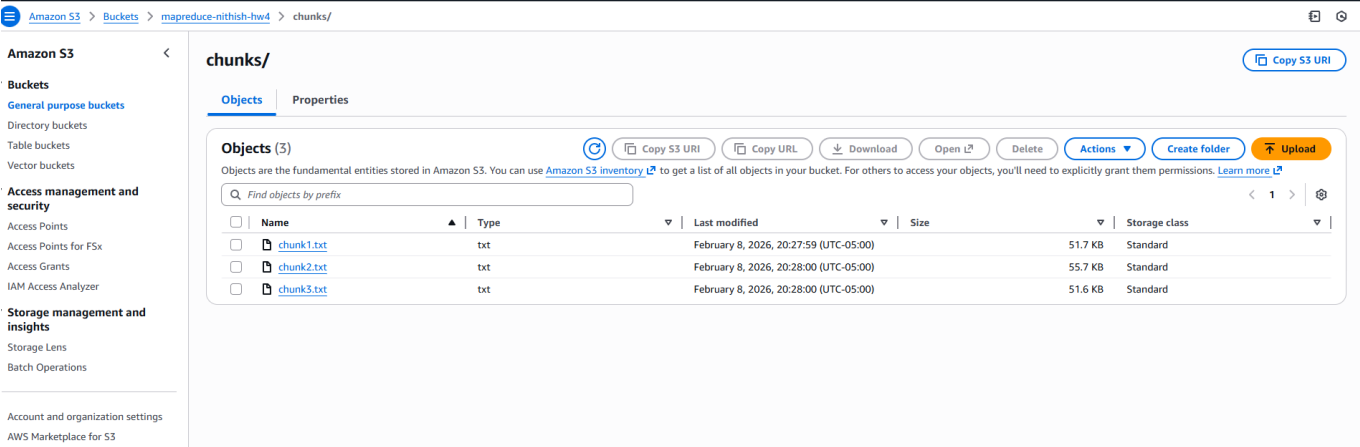
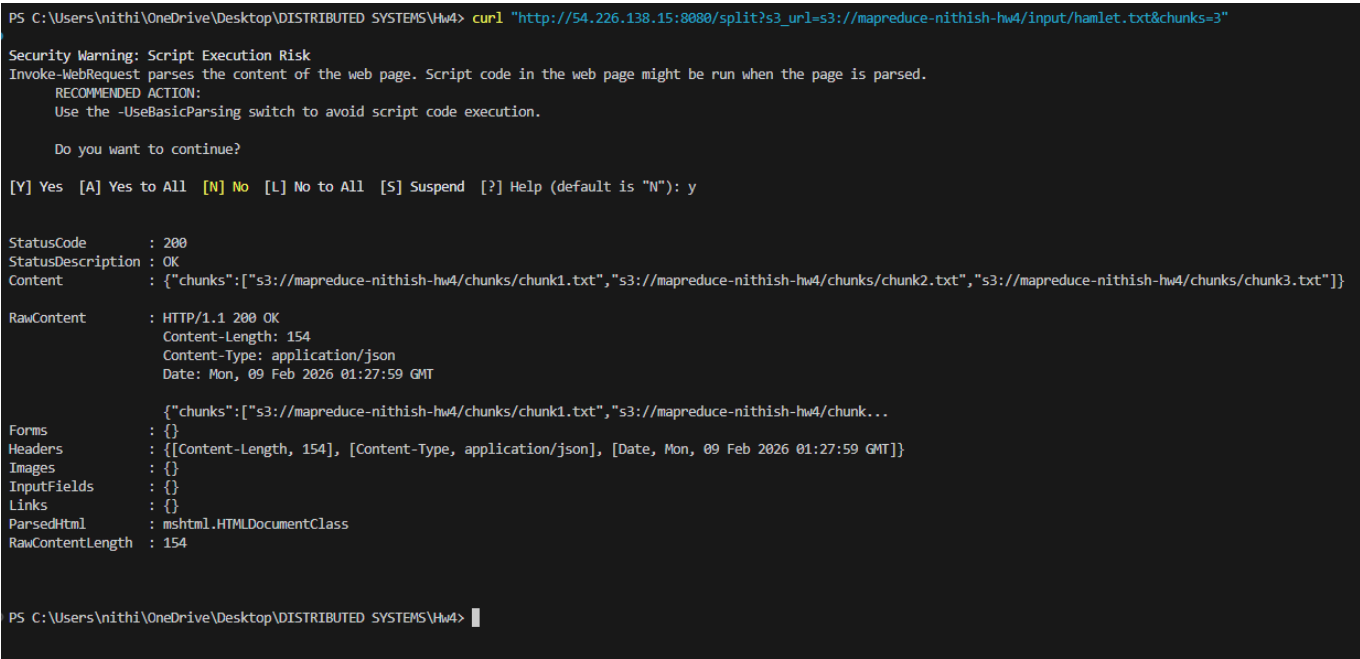
```
http://<splitter-ip>:8080/split?s3_url=s3://mapreduce-nithish-hw4/input/hamlet.txt&chunks=3
```

Response

```
{
  "chunks": [
    "s3://mapreduce-nithish-hw4/chunks/chunk1.txt",
    "s3://mapreduce-nithish-hw4/chunks/chunk2.txt",
    "s3://mapreduce-nithish-hw4/chunks/chunk3.txt"
  ]
}
```

The splitter successfully:

- Read the input file from S3
- Split it into three roughly equal-sized chunks
- Uploaded each chunk back to S3



- Curl output showing splitter response
- S3 `chunks/` folder with `chunk1.txt`, `chunk2.txt`, `chunk3.txt`

Step 4: Mapper Task Execution

Three mapper tasks were executed independently, each processing one chunk.

Each mapper:

- Read its assigned chunk from S3
- Counted word occurrences
- Wrote results as a JSON file to S3

Example Mapper Request

```
http://<mapper-ip>:8080/map?s3_url=s3://mapreduce-nithish-hw4/chunks/chunk1.txt
```

Mapper Output

```
{  
  "result": "s3://mapreduce-nithish-hw4/mapper-results/chunk1.json"  
}
```

After running all three mappers, the following files were produced:

```
s3://mapreduce-nithish-hw4/mapper-results/chunk1.json  
s3://mapreduce-nithish-hw4/mapper-results/chunk2.json  
s3://mapreduce-nithish-hw4/mapper-results/chunk3.json
```

```

PS C:\Users\nithi\OneDrive\Desktop\DISTIBUTED SYSTEMS\hw4> curl "http://54.152.54.160:8080/map?s3_url=s3://mapreduce-nithish-hw4/chunks/chunk1.txt"

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): A

StatusCode      : 200
StatusDescription : OK
Content         : {"result":"s3://mapreduce-nithish-hw4/mapper-results/chunk1.json"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 67
                  Content-Type: application/json
                  Date: Mon, 09 Feb 2026 01:31:18 GMT

                  {"result":"s3://mapreduce-nithish-hw4/mapper-results/chunk1.json"}

Forms           : {}
Headers         : {[Content-Length, 67], [Content-Type, application/json], [Date, Mon, 09 Feb 2026 01:31:18 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 67

PS C:\Users\nithi\OneDrive\Desktop\DISTIBUTED SYSTEMS\hw4>

```

Amazon S3 > Buckets > mapreduce-nithish-hw4 > mapper-results/

mapper-results/ Copy S3 URI

Objects (3)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
chunk1.json	json	February 8, 2026, 20:31:19 (UTC-05:00)	23.4 KB	Standard
chunk2.json	json	February 8, 2026, 20:32:30 (UTC-05:00)	26.1 KB	Standard
chunk3.json	json	February 8, 2026, 20:32:35 (UTC-05:00)	23.6 KB	Standard

- Curl output for mapper invocation
- S3 `mapper-results/` folder showing all three JSON files
- CLI `aws s3 ls` output verifying mapper results

Step 5: Reducer Task Execution

The reducer task was invoked with the three mapper result URLs as query parameters.

Request

```

http://<reducer-ip>:8080/reduce?url=s3://mapreduce-nithish-hw4/mapper-
results/chunk1.json
&url=s3://mapreduce-nithish-hw4/mapper-results/chunk2.json
&url=s3://mapreduce-nithish-hw4/mapper-results/chunk3.json

```

Response

```
{
  "result": "s3://mapreduce-nithish-hw4/results/final-wordcount.json"
}
```

The reducer:

- Read all mapper outputs from S3
- Aggregated word counts
- Wrote the final combined result back to S3

```
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4> aws s3 ls s3://mapreduce-nithish-hw4/mapper-results/
2026-02-08 20:31:19      23983 chunk1.json
2026-02-08 20:32:30      26734 chunk2.json
2026-02-08 20:32:35      24182 chunk3.json
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4> curl "http://100.27.3.254:8080/reduce?url=s3://mapreduce-nithish-hw4/mapper-results/chunk1.json&url=s3://mapreduce-nithish-hw4/mapper-results/chunk2.json&url=s3://mapreduce-nithish-hw4/mapper-results/chunk3.json"

StatusCode      : 200
StatusDescription : OK
Content         : {"result":"s3://mapreduce-nithish-hw4/results/final-wordcount.json"}
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 69
                  Content-Type: application/json
                  Date: Mon, 09 Feb 2026 01:35:07 GMT

                  {"result":"s3://mapreduce-nithish-hw4/results/final-wordcount.json"}
Forms           : {}
Headers         : [[Content-Length, 69], [Content-Type, application/json], [Date, Mon, 09 Feb 2026 01:35:07 GMT]]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 69

PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4>
```

```
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4> aws s3 ls s3://mapreduce-nithish-hw4/results/
2026-02-08 20:35:08      53776 final-wordcount.json
• PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4> aws s3 cp s3://mapreduce-nithish-hw4/results/final-wordcount.json .
  download: s3://mapreduce-nithish-hw4/results/final-wordcount.json to .\final-wordcount.json
○ PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw4>
```

- Curl output for reducer request
- S3 **results/** folder showing **final-wordcount.json**

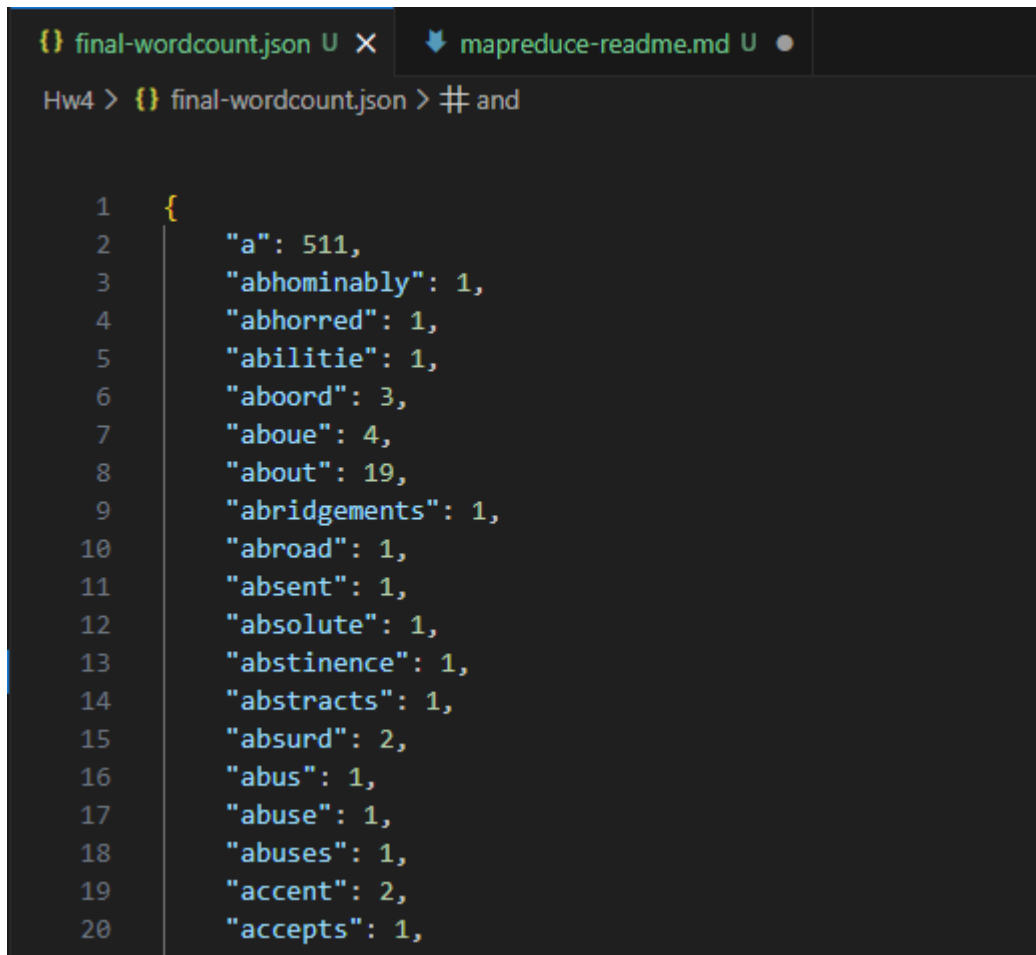
Step 6: Result Verification

The final result file was downloaded locally and formatted using VS Code for readability.

Final Output Location

```
s3://mapreduce-nithish-hw4/results/final-wordcount.json
```

The JSON correctly reflects aggregated word counts across all chunks, validating the correctness of the MapReduce pipeline.



```
1  {
2    "a": 511,
3    "abominably": 1,
4    "abhorred": 1,
5    "abilitie": 1,
6    "aboord": 3,
7    "aboue": 4,
8    "about": 19,
9    "abridgements": 1,
10   "abroad": 1,
11   "absent": 1,
12   "absolute": 1,
13   "abstinence": 1,
14   "abstracts": 1,
15   "absurd": 2,
16   "abus": 1,
17   "abuse": 1,
18   "abuses": 1,
19   "accent": 2,
20   "accepts": 1,
```

- Beautified `final-wordcount.json` opened in VS Code

Observations and Learnings

- The workload was embarrassingly parallel, making it ideal for MapReduce.
- ECS Fargate made scaling trivial by running multiple mappers independently.
- S3 acted as a reliable shared storage medium for coordination.
- Manual orchestration highlighted the complexity of scheduling and fault handling in distributed systems.

The reducer correctly aggregates mapper outputs; however, comparison with a centralized baseline revealed discrepancies due to differences in tokenization and chunk-boundary effects. Words containing apostrophes such as 'tis and o'er were split differently by the distributed mappers, leading to variations in counts. These inconsistencies highlight an important distributed systems lesson: preprocessing must be globally consistent to guarantee identical results.

We compared a single-machine word count implementation with our distributed MapReduce-style implementation using ECS tasks and S3.

For hamlet.txt, the local version completed in 10.14 ms, while the distributed version took 1045 ms, a slowdown of approximately 103×.

For a larger input (big_10x.txt), the local version took 109.32 ms, while the distributed version took 1416.71 ms, reducing the slowdown to approximately 13×.

This demonstrates that for small to moderate datasets, distributed execution is dominated by orchestration, network, and storage overhead. However, as input size grows, the relative overhead decreases, illustrating the scalability motivation behind MapReduce.